

A Short Guide to This Vault

What this Obsidian vault is, how the daily loop works,
and why it asks you to write down a minimum for everything.

This vault is a personal operating system built from plain markdown files and an AI coding agent working alongside you inside them. It is not a note-taking app with extra steps, and it is not a task manager either. It is closer to a small piece of infrastructure: a place where a day gets planned, a day gets closed out, and the things you learn while doing that get compiled into something you can actually search again later, instead of evaporating into scrollback.

The rest of this guide walks through the three zones the vault is split into, the loop that repeats every day, the pipeline that turns raw material into a wiki, and the one rule that ties all of it together: every block of time gets a minimum, and minimum done is the only thing that counts as a finished day.

The three zones

Everything in the vault lives in one of three top-level folders, and each one answers a different question.

Life

Life is where today happens. The calendar, the daily journal, the inbox of loose tasks, and the running log of how the day actually went, as opposed to how it was planned. This folder changes every single day and is meant to. Nothing in here is supposed to still be relevant in six months in its current form -- it gets read, acted on, and folded into the next day's plan.

Library

Library is where things you learned get kept. It does not hold raw clippings forever -- it holds compiled pages: one page per real mechanism, each one tracing back to the specific source it came from. Library changes rarely and is meant to accumulate instead of getting rewritten. The test for whether something belongs here is simple: would you want to find this again in three months without remembering which day you learned it?

System

System is the machinery underneath both of the above: templates, saved conversation checkpoints, and the configuration that makes the daily loop and the Library pipeline actually run. You do not visit System very often, but Life and Library both depend on it quietly in the background.

The zones are not walls. An agent working in Life can read Library's index to know what you already understand, and Library can read Life's plan to know what you are currently working toward. What they do not do is write into each other's space by accident -- Life never rewrites a wiki page, and a Library session never edits tomorrow's calendar.

The daily loop

A day in this system has a fixed shape, made of four moves that repeat every single day whether or not the day itself goes well.

1. Morning

The day opens by reading whatever got planned the night before: the blocks on today's calendar and their minimums. Nothing new gets invented here -- the point is to see the day before it starts, not to replan it from scratch every morning. If something genuinely has to move, it moves, but that is the exception, not the routine.

2. Blocks

The middle of the day is just the blocks happening, or not happening. Each block was written with a concrete minimum attached to it -- not a vague intention like 'work on the project,' but something that can be checked when the block is over: a test that passes, a page that got read, a shift that got worked. Empty stretches of calendar are left empty on purpose. A vault that fills every hour with filler just to look busy stops telling you anything useful about how the day actually went.

3. Evening

The day closes by looking honestly at what happened against what was planned: which blocks landed, which didn't, and why. This is also where tomorrow gets its first draft -- a short planning pass, deliberately kept small, so it happens every single night instead of turning into a project of its own.

4. Patterns

One small, factual entry gets appended to a running log every day: what time things started, what got interrupted, what worked. A single day is noise. The same thing showing up three separate times is a pattern worth actually planning around -- so the log waits for repetition before it promotes anything to a conclusion. This is what turns a calendar into something that gets smarter about you over time instead of just recording time.

The Library pipeline

Learning something and having a searchable page about it are two different events, and the gap between them is exactly where most notes systems quietly fail. This vault treats that gap as a pipeline with four distinct steps, so nothing has to happen all at once.

1. Clip

Raw material comes in however it naturally shows up: an article worth keeping, a note written by hand after solving a real bug, a conversation that actually explained something. None of it is edited or judged at this stage -- it is just captured, as close to verbatim as possible.

2. Inbox

Every new piece of raw material lands in one single queue. It waits there, unmodified, until someone deliberately decides it is worth compiling. A queue that never gets touched is not a failure of the system -- it is honest information that says something wasn't actually worth the next step, and that is fine.

3. Compile

This is the only step that costs real attention, and it happens on purpose, not automatically. One source at a time gets read closely and asked a blunt question: does this contain an actual mechanism -- something you could explain to someone else, something with a 'why,' not just a fact -- or is it just a pointer to something you might want later? A mechanism earns its own page. A pointer earns one line in a list of pointers. Nothing gets a page just because it existed.

4. Wiki and the map

A compiled page always keeps a visible link back to the exact source it came from -- no page in this Library is allowed to exist without a receipt. Pages are grouped into folders by topic, and every folder has one entry-point page, a map of that folder, written in the order you would actually want to learn the material rather than alphabetically. Finding the map is finding the whole topic; a page that isn't reachable from its folder's map might as well not exist, because nobody will ever stumble onto it again.

Why minimums

The single rule that shows up everywhere in this vault -- on every calendar block, in every evening review -- is the requirement that a block of time comes with a minimum: a specific, checkable thing that counts as this block having happened at all.

The alternative is familiar to anyone who has ever planned a day around a vague intention. "Work on the project" for two hours can mean genuinely making progress, or it can mean two hours of reading around the topic without writing a line of code, and at the end of the day both of those get remembered the same way: 'I worked on the project.' A minimum removes that ambiguity before the block even starts. Either the test passes or it doesn't. Either the chapter got read or it didn't. There is no comfortable middle ground to retreat into at the end of the day.

This is also why a day with three small tasks done and one important block skipped does not automatically count as a good day just because three things got checked off. The minimum belongs to the block that actually mattered, and if that one didn't happen, the day's honest verdict is that it didn't happen -- not 'mostly,' not 'basically.' A system that lets you round up every close call stops being useful exactly at the moment you need it to be useful.

None of this is about being harsh for its own sake. It's about making sure the record of a week actually matches the week, so that when a pattern shows up in that record -- a block that always gets skipped, an hour of the day that's consistently dead -- it's a real pattern and not an artifact of everyone grading their own homework generously. The whole point of keeping this system running every day is to eventually trust what it tells you.

That's the whole shape of it: three zones, a daily loop, a compile pipeline, and one rule about minimums holding all of it together. The next two sections work through what a first day looks like in practice, and the questions that tend to come up once you're actually using it.

A first day, walked through

It helps to see the loop from the previous section as one ordinary day instead of a diagram. Nothing about this day is special -- that's the point.

Before anything else

A few calendar blocks already exist from the night before, each with a minimum attached: a study block, a work shift, a block for whatever side project is currently active. The morning routine reads these out loud, in effect -- what's first, what it's for, and which one block matters most if the day goes sideways and only one thing gets done.

During the day

The blocks happen, mostly on schedule and sometimes not. A block running long pushes the next one back; a block that turns out to be pointless gets cut short on purpose rather than sat through out of obligation. None of this gets fixed in the moment -- it gets written down honestly at the end of the day instead, which is a calmer place to look at it from.

At the end of the day

The evening pass is short by design: what got done, what didn't and why, one sentence of honest verdict on whether the day's minimum was actually met, and then a first draft of tomorrow. If something was learned along the way -- a bug that took an hour to understand, an article that actually explained a confusing term -- that raw material gets dropped into the Library's inbox in passing, to be compiled properly some other time. It does not get written up in full on the same night the day itself is being closed out; that would turn a ten-minute habit into an hour-long one, and an hour-long habit doesn't survive a busy week.

Repeat that daily, and after a few weeks two things exist that didn't before: a written record of how the days actually went, not how they were supposed to go, and a small queue of raw material waiting for the moment it's worth turning into something permanent.

Common questions

What happens if a day gets skipped entirely?

Nothing dramatic. The next day's loop just starts from wherever things actually are, not from where the plan assumed they'd be. A missed day is data, not a reason to abandon the whole week -- the daily log is more useful with an honest gap in it than with a day quietly backfilled to look better than it was.

What if something in the inbox queue never gets compiled?

That's allowed, and it's informative rather than a failure. A queue with a lot sitting in it and nothing being compiled is itself a signal worth noticing -- it usually means there isn't currently time for this part of the loop, which is more useful to know honestly than to paper over with a rushed, shallow compile just to empty the queue.

Why isn't the whole pipeline automated?

Clipping and queueing are mechanical, so they're automatic. Deciding whether something contains a real, explainable mechanism is a judgment call, and making that call is most of the value of the compile step -- automate it away and the wiki fills up with pages nobody actually needed, which is worse than a smaller wiki that only holds things someone deliberately decided were worth keeping.

Why keep minimums instead of just tracking hours spent?

Hours spent measures effort, not whether anything happened. Two hours can produce a working feature or produce nothing at all, and a system that only counts hours can't tell those two outcomes apart. A minimum is the smallest honest answer to "did this block actually happen," which is a more useful question to have answered at the end of a week than "how much time went by."